

The `for book` loop iterates over the `books` list, which requires n steps multiplied by the steps inside the loop. This loop includes a nested `for i` loop, which iterates 100 times. This means the `for book` loop runs $102 \times n$, or $102n$ steps. After dropping the coefficient, we find that `cheerForFavoriteBook()` is still just an $O(n)$ linear operation. This 102 coefficient might seem rather large to just ignore, but consider this: if `favorite` never appears in the `books` list, this function would only run $1n$ steps. The impact of coefficients can vary so wildly that they aren't very meaningful.

Next, the `findDuplicateBooks()` function searches through the `books` list (a linear operation) once for each book (another linear operation):

```
def findDuplicateBooks(books):
    for i in range(books): # n steps
        for j in range(i + 1, books): # n steps
            if books[i] == books[j]: # 1 step
                print('Duplicate:', books[i]) # 1 step
```

The `for i` loop iterates over the entire `books` list, performing the steps inside the loop n times. The `for j` loop also iterates over a portion of the `books` list, although because we drop coefficients, this also counts as a linear time operation. This means the `for i` loop performs $n \times n$ operations—that is, n^2 . This makes `findDuplicateBooks()` an $O(n^2)$ polynomial time operation.

Nested loops alone don't imply a polynomial operation, but nested loops where both loops iterate n times do. These result in n^2 steps, implying an $O(n^2)$ operation.

Let's look at a challenging example. The binary search algorithm mentioned earlier works by searching the middle of a sorted list (we'll call it the `haystack`) for an item (we'll call it the `needle`). If we don't find the `needle` there, we'll proceed to search the previous or latter half of the `haystack`, depending on which half we expect to find the `needle` in. We'll repeat this process, searching smaller and smaller halves until either we find the `needle` or we conclude it isn't in the `haystack`. Note that binary search only works if the items in the `haystack` are in sorted order.

```
def binarySearch(needle, haystack):
    if not len(haystack): # 1 step
```